Type Erasure in C++

Heterogeneous types without intrusive inheritance

Antoine MORRIER

01

The Problem

How do we work with different types in a uniform way?

A simple question

We have unrelated types that share a common behavior...

```
struct Circle {
    double radius;
    void draw() const { std::cout << "Circle(" << radius << ")\n"; }
};

struct Square {
    double side;
    void draw() const { std::cout << "Square(" << side << ")\n"; }
};
```

How do we store them together?

```
std::vector<???> shapes;
```

This problem is everywhere

- **Drawable collection** — render different shapes through the same interface
- **Callbacks** — store lambdas, function pointers, and functors in one container
- **Plugin system** — load and call modules that share no common base class
- **Event handlers** — register heterogeneous handlers for the same event

We need a way to manipulate unrelated types uniformly.

02

Existing Solutions

What tools does C++ already give us?

Inheritance + Virtual

```
struct Shape {
    virtual ~Shape() = default;
    virtual void draw() const = 0;
};
```

```
struct Circle : Shape {
    double radius;
    void draw() const override {
        std::cout << "Circle(" << radius << ")\n";
    }
};
```

```
struct Square : Shape {
    double side;
    void draw() const override {
        std::cout << "Square(" << side << ")\n";
    }
};
```

```
std::vector<std::unique_ptr<Shape>> shapes;
```

Inheritance — The Downsides

- **Intrusive** — every type must inherit from `Shape`
- **Can't adapt third-party types** — you don't own their code
- **Requires pointers** — `unique_ptr` or `shared_ptr` everywhere
- **No value semantics** — can't simply copy a `vector<unique_ptr<Shape>>`. If `shared_ptr`, modifying one will contaminates all the others.
- **Pointers invite bugs** — dangling, slicing, ownership confusion

void* — "Don't do this"

```
struct Wrapper {
    void* data;
    void (*draw_fn)(const void*);
};

void draw_circle(const void* p) {
    static_cast<const Circle*>(p)->draw();
}
```

- No type safety — cast to the wrong type = **undefined behavior**
- No ownership semantics — who deletes what?
- Compiler can't help you

Never do this in modern C++.

std::variant

```
using Shape = std::variant<Circle, Square>;

std::vector<Shape> shapes;
shapes.push_back(Circle{2.0});
shapes.push_back(Square{3.0});

for (auto& s : shapes) {
    std::visit([](auto& shape) { shape.draw(); }, s);
}
```

- **Closed set** — every type must be listed at compile time
- **Visitor is verbose** — gets unwieldy with many types
- Adding a new type requires modifying the variant definition

Comparison

	Non-intrusive	Type-safe	Open set	Value semantics	No pointer needed
Inheritance	No	Yes	Yes	No	No
void*	Yes	No	Yes	No	No
variant	Yes	Yes	No	Yes	Yes
Type Erasure	**Yes**	**Yes**	**Yes**	**Yes**	**Yes**

03

Type Erasure in the Standard

Already in your code

std::function

```
// Lambda
std::function<int(int, int)> add = [](int a, int b) { return a + b; };

// Function pointer
int multiply(int a, int b) { return a * b; }
std::function<int(int, int)> mul = multiply;

// Functor
struct Divider {
    int operator()(int a, int b) { return a / b; }
};
std::function<int(int, int)> div = Divider{};
```

One type, **three different callables**.

Internally: concept/model pattern + small buffer optimization.

std::any

```
std::any a = 42;
std::cout << std::any_cast<int>(a) << "\n";        // 42

a = std::string("hello");
std::cout << std::any_cast<std::string>(a) << "\n"; // hello

a = Circle{5.0};
std::any_cast<Circle>(a).draw();                    // Circle(5)
```

- Stores **any type** with value semantics
- **any_cast** throws **bad_any_cast** on type mismatch
- **No interface**— you must know the type to extract it

The Concept / Model Pattern

Wrapper — public API, value semantics

Concept — private abstract interface

Model<T> — template that wraps any conforming type

```
class Drawable {
  // public draw()
  // stores unique_ptr<Concept>

  struct Concept {
    virtual void draw() = 0;
  };

  template<typename T>
  struct Model : Concept {
    T obj;
    void draw() {
      obj.draw();
    }
  };
};
```

Let's build this step by step.

04

Building Type Erasure

5 steps: from inheritance to type erasure

Step 1 — Classic Inheritance

```
struct Shape {
    virtual ~Shape() = default;
    virtual void draw() const = 0;
};

struct Circle : Shape {
    double radius;
    void draw() const override {
        std::cout << "Circle(" << radius << ")\n";
    }
};

struct Square : Shape {
    double side;
    void draw() const override {
        std::cout << "Square(" << side << ")\n";
    }
};
```

```
std::vector<std::unique_ptr<Shape>> shapes;
shapes.push_back(std::make_unique<Circle>(Circle{2.0}));
shapes.push_back(std::make_unique<Square>(Square{3.0}));
```

Problem: intrusive, requires pointers, no value semantics.

Step 2 — Extract the Concept

Move the interface *inside* a wrapper class.

```
class Drawable {
    struct Concept {
        virtual ~Concept() = default;
        virtual void draw() const = 0;
    };

    std::unique_ptr<Concept> self;
};
```

Key insight: the abstract base class is now a *private implementation detail*.
External types don't need to know about it.

Step 3 — Create Model<T>

A template that wraps *any* type with a `draw()` method.

```
class Drawable {
    struct Concept {
        virtual ~Concept() = default;
        virtual void draw() const = 0;
    };

    template <typename T>
    struct Model : Concept {
        T obj;

        Model(T obj) : obj(std::move(obj)) {}

        void draw() const override {
            obj.draw();
        }
    };

    std::unique_ptr<Concept> self;
};
```

Model<T> bridges any concrete type to the internal Concept.

Step 4 — Template Constructor + Public API

```
class Drawable {
    struct Concept { /* ... */ };

    template <typename T>
    struct Model : Concept { /* ... */ };

    std::unique_ptr<Concept> self;

public:
    template <typename T>
    Drawable(T obj)
        : self(std::make_unique<Model<T>>(std::move(obj))) {}

    void draw() const {
        self->draw();
    }
};
```

The constructor **deduces T** automatically — any type with `draw()` just works.

Step 4 bis — Heterogeneous Collection

```
// No inheritance needed!
struct Circle {
    double radius;
    void draw() const { std::cout << "Circle(" << radius << ")\n"; }
};

struct Square {
    double side;
    void draw() const { std::cout << "Square(" << side << ")\n"; }
};
```

```
// Value semantics, no pointers in user code!
std::vector<Drawable> shapes;
shapes.push_back(Circle{2.0});
shapes.push_back(Square{3.0});

for (auto& s : shapes) {
    s.draw();
}
```

No base class, no pointers, no variant — just values.

Step 5 — Copy Semantics

`unique_ptr` is move-only. We need `clone()` for deep copy.

```cpp
struct Concept {
    virtual ~Concept() = default;
    virtual void draw() const = 0;
    virtual std::unique_ptr<Concept>
        clone() const = 0;
};

template <typename T>
struct Model : Concept {
    T obj;
    Model(T obj) : obj(std::move(obj)) {}
    void draw() const override {
        obj.draw();
    }

    std::unique_ptr<Concept>
    clone() const override {
        return std::make_unique<Model>(obj);
    }
};
```

```cpp
// Copy constructor & assignment
Drawable(const Drawable& other)
    : self(other.self->clone()) {}

Drawable& operator=(const Drawable& other)
{
    self = other.self->clone();
    return *this;
}

// We force move semantic
Drawable(Drawable&&) noexcept = default;
Drawable& operator=(Drawable&&) noexcept
    = default;
```

Complete Implementation

```
class Drawable {
    struct Concept {
        virtual ~Concept() = default;
        virtual void draw() const = 0;
        virtual std::unique_ptr<Concept> clone() const = 0;
    };

    template <typename T>
    struct Model : Concept {
        T obj;
        Model(T obj) : obj(std::move(obj)) {}
        void draw() const override { obj.draw(); }
        std::unique_ptr<Concept> clone() const override { return std::make_unique<Model>(obj); }
    };

    std::unique_ptr<Concept> self;

public:
    template <typename T>
    Drawable(T obj) : self(std::make_unique<Model<T>>(std::move(obj))) {}

    Drawable(const Drawable& other) : self(other.self->clone()) {}
    Drawable& operator=(const Drawable& other) { self = other.self->clone(); return *this; }
    Drawable(Drawable&&) = default;
    Drawable& operator=(Drawable&&) = default;

    void draw() const { self->draw(); }
};
```

~30 lines. Non-intrusive, type-safe, value semantics, open set.

Small Buffer Optimization (SBO)

Without SBO

Every `Drawable` allocates on the heap via `make_unique`.

```
// Without SBO
std::unique_ptr<Concept> self;
// heap alloc every time
```

With SBO

Small objects are stored *inline* in a fixed-size buffer, avoiding allocation.

```
// With SBO
alignas(...) char buf[N];
Concept* self;
// placement new if sizeof(T) <= N
// heap alloc otherwise
```

`std::function` and `std::any` both use SBO internally.

We won't implement it here — just know it exists and matters for performance.

Constraining with Concepts (C++20)

Without constraints, passing a wrong type produces *unreadable* errors deep inside `Model<T>`.

```
// Define the concept
template <typename T>
concept HasDraw = requires(const T& t) {
    t.draw();
};
```

```
// Constrain the constructor
template <HasDraw T>
Drawable(T obj)
    : self(std::make_unique<Model<T>>(
        std::move(obj))) {}
```

```
struct NotDrawable { int x; };
Drawable d{NotDrawable{}};
```

Before: Wall of errors inside `Model<T>::draw()`
"no member named 'draw' in 'NotDrawable'" — buried in template instantiation.

After: Clear, immediate message
"constraints not satisfied for 'Drawable(T)' [with T = NotDrawable]"

Concepts make the interface explicit and errors actionable.

05

C++26: std::polymorphic

You won't need to write this yourself anymore

std::polymorphic<Shape>

Value semantics + polymorphism, built into the standard. *(P3019)*

```
struct Shape {
    virtual ~Shape() = default;
    virtual void draw() const = 0;
};

struct Circle : Shape {
    double radius;
    void draw() const override {
        std::cout << "Circle(" << radius << ")\n";
    }
};
```

```
std::vector<std::polymorphic<Shape>> shapes;
shapes.push_back(Circle{2.0});
shapes.push_back(Square{3.0});

auto copy = shapes; // deep copy, just works!
```

Copyable, movable, no `unique_ptr`, no slicing.

Our Drawable vs std::polymorphic

Our Drawable (~30 lines)

```
class Drawable {
    struct Concept { /* ... */ };
    template<typename T>
    struct Model : Concept { /* ... */ };
    std::unique_ptr<Concept> self;
public:
    template<typename T>
    Drawable(T obj);
    void draw() const;
    // + copy/move
};
```

Duck-typed: any type with `draw()`

std::polymorphic

```
std::polymorphic<Shape> shape = Circle{2.0};

// That's it.
// Deep copy, value semantics,
// polymorphism — all built-in.
```

Requires a base class (inheritance)

	Our Drawable	std::polymorphic
Non-intrusive	Yes (duck-typed)	No (needs base)
Standard library	No (custom)	Yes (C++26)
Value semantics	Yes	Yes

std::indirect<T>

Also from P3019 — value semantics without polymorphism.

```
// Like unique_ptr but with value semantics
std::indirect<std::string> name = "hello";

auto copy = name; // deep copy
*copy = "world";

std::cout << *name << "\n";  // "hello" — independent copy
std::cout << *copy << "\n";  // "world"
```

`std::indirect<T>`	Value semantics, **no polymorphism**
`std::polymorphic<T>`	Value semantics **+ polymorphism**

06

Conclusion

Key Takeaways

1. **The problem is real** — heterogeneous collections are everywhere in C++
2. **Inheritance is intrusive** — it couples types to a hierarchy
3. **Type erasure = Concept + Model** — hide the interface, wrap any conforming type
4. **Value semantics matter** — clone pattern gives deep copy without pointers in user code
5. **C++26 brings std::polymorphic** — the standard solution for polymorphic value types

Resources

- **Sean Parent** — *"Inheritance Is The Base Class of Evil"* (GoingNative 2013)
- **Klaus Iglberger** — *"Breaking Dependencies: Type Erasure"* (CppCon)
- **P3019** — *"std::indirect and std::polymorphic"* (C++26 proposal)

Questions?

Antoine MORRIER